

DERLEYİCİ VE ÇALIŞTIRMA ORTAMI

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler** (**compiler**) tarafından makine koduna çevrilerek **icra edilebilir** (**executable**) dosya olarak kaydedilebilmektedir.

C# dilinde programlama öğrenmeye başlamak için ilk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları ***.cs** uzantısı ile dosya olarak saklamaktır. İkinci adım ise .Net Yazılım Çerçevesinin (Kısaca **.Net Framework**) uygun sürümünü **işletim sisteminizde** (**operating system**) kurmaktır. Üçüncü adım ise yazdığınız kodu derleyerek çalışabilen bir icra edilebilir dosyayı oluşturup çalıştırmaktır. Derleyici için girdi, kaynak kodu dosyasıdır. Kaynak dosyasında yazılan kaynak kodu, içerisinde C# **talimatları** (**statement**) olan, insan tarafından okunabilen metin dosyasıdır.

.NET Framework

.NET Framework, platformun çalışan uygulamalarına çeşitli hizmetler sağlayan bir **çalıştırma ortamıdır** (**run-time**). İki ana bileşenden oluşur:

1. **Base Class Library (BCL)**: Geliştiricilerin kendi uygulamalarından çağırabileceği, test edilmiş ve yeniden kullanılabilir bir kitaplık olan **.NET Framework sınıf kitaplığıdır** (**.Net Class Library**).
2. **Common Language Runtime (CLR)**: Uygulamaları çalıştıran ve yöneten motordur (engine) yani **ortak dil çalışma ortamıdır**. İş parçacığı (**thread**) yönetimi, **çöp toplama** (**garbage collection**), **tip güvenliği** (**type safety**), **istisna işleme** (**exception handling**) ve daha fazlası gibi hizmetler sağlar.

.NET Framework çalışan uygulamalar için sağlanan hizmetleri sunar:

1. Bellek yönetimi: Birçok programlama dilinde, programcılar bellek ayırmayı ve serbest bırakmayı ve nesne ömrünü işlemekten sorumludur. .NET Framework uygulamalarda, CLR bu hizmetleri uygulama adına sağlar.
2. Ortak tip sistemi (**Common Type System-CTS**): Geleneksel programlama dillerinde temel tipler derleyici tarafından tanımlanır, bu da başka dillerde yazılan uygulamaların ortak çalışabilirliği karmaşılaştırır. .NET Framework, temel tipleri ortak tip sistemi tarafından tanımlanır ve .NET Framework ile çalışan tüm dillerde ortaktır.
3. Kapsamlı bir sınıf kitaplığı: Programcılar, sık kullanılan alt düzey programlama işlemlerini işlemek için çok miktarda kod yazmak yerine, .NET Framework sınıf kitaplığından bir tür ve üyeleri olan kolay erişilebilir bir kitaplık kullanır.
4. Geliştirme çerçeveleri: .NET Framework, web uygulamaları için ASP.NET, veri erişimi için ADO.NET, hizmet odaklı uygulamalar için Windows Communication Foundation (WCF).
5. Dil birlikte çalışabilirliği: .NET Framework hedef olan dil derleyicileri, kaynak kodları **ortak ara dile** (**Common Intermediate Language-CIL**) derler ve CLR tarafından çalışma zamanında derlenir. Bu özellik ile, bir dilde yazılan yöntemler diğer diller tarafından erişilebilir olur. Böylece programcılar tercih ettiği dilde uygulama oluşturmaya odaklanmaktadır.
6. Sürüm uyumluluğu: Nadir özel durumlar dışında belirli bir .NET Framework sürümü kullanılarak geliştirilen uygulamalar, daha sonraki bir sürümde değişiklik yapılmadan çalışır.
7. Yan yana çalıştırma: .NET Framework, aynı bilgisayarda ortak dil çalışma ortamının birden çok sürümünün var olmasına izin vererek sürüm çakışmalarını çözmeye yardımcı olur. bu, birden çok uygulama sürümünün ve bir uygulamanın oluşturulduğu .NET Framework sürümünde çalışabileceği anlamına gelir.
8. Çoklu sürüm desteği: Geliştiriciler standart bir sürümü tarafından desteklenen birden çok .NET Framework platformda çalışan sınıf kitaplıkları oluşturur.

Ortak Dil Tarifnamesi

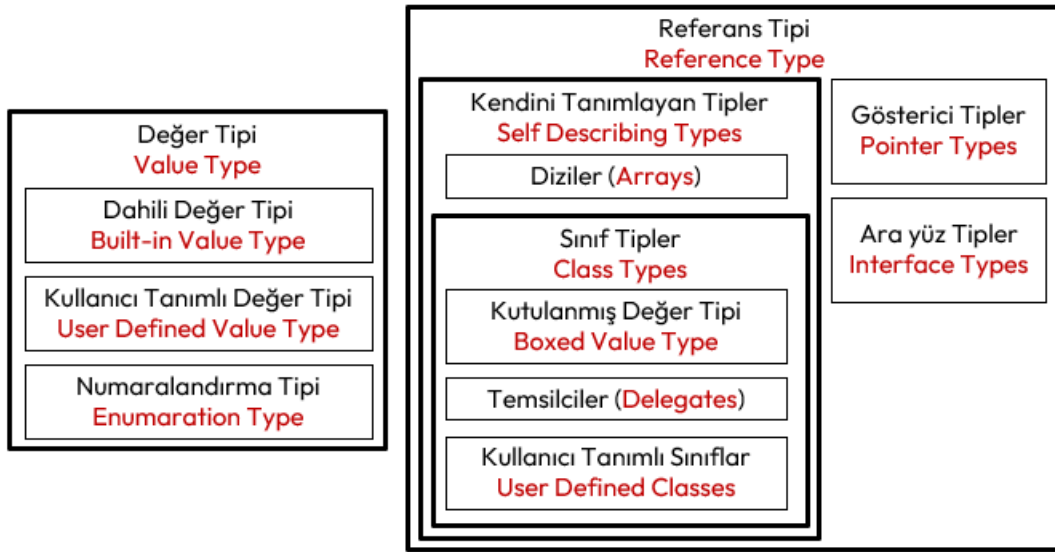
Ortak Dil Tarifnamesi (**Common Language Specification-CLS**) programlama dillerinin nesne modellemesinden kaynaklanan farklılıklarını bertaraf etmek için geliştirilmiş standartlar dizisidir. Bunlar aşağıda belirtilen başlıklarda toplanmaktadır;

- **Yapı** (**structure**) standardı.
- **Ortak ara dil** (**Common Intermediate Language-CIL**): Programlama dili ne olursa olsun, bu diller herkes tarafından bilinecek bir ara koda dönüştürülür ya da derlenir.
- **Veri tipi standardı** (**Common Type System-CTS**) aşağıda anlatılmıştır.
- **Olay** (**event**) standartları.
- Verilerin **bellekteki yerleşimine** (**persistence**) ilişkin standartlar: Aynı blok içinde tanımlanan **char**, **int**, **double** değişkenlerinin tanımlandığı örneği göz önüne alacak olursak; C dilinde, belleğe ilk önce **char**, sonra **int**, daha sonra **double** yerleşir (**row major**). Pascal dilinde ise ilk önce **double**, sonra **int**, son olarak da **char** yerleşir (**column major**).

İşte bu tip değişken yerleşimlerinden kaynaklanacak problemleri gidermek için bu standartlar oluşturulmuştur.

Ortak Tip Sistemi

Ortak Tip Sistemi (**CTS**), dillerdeki veri tipleri arasındaki farklılıkların bertaraf edilmesine yönelik standartlardır. Bu sistem ile bir dilde 4 bayt, diğer bir dilde 8 bayt olan gerçek sayı tanımlaması gibi karmaşıklıklar giderilmiştir.



Şekil 4. Ortak Tip Sistemi (CTS)

CTS, aşağıdaki veri tiplerini desteklemektedir:

1. **Değer Tipleri** (**value type**)
 - a) **Dâhili değer tipleri** (**built-in value type**): **bool**, **integer**, **byte**, **double**, **char** gibi veri tipleridir.
 - b) **Kullanıcı tanımlı değer tipleri** (**user-defined value type**): **struct** gibi kullanıcı tarafından, yerleşik değer tiplerinin çeşitli şekillerde bir araya getirilip tek bir isim altında toplanmasıyla oluşan tiplerdir.
 - c) **Numaralandırma** (**enumeration**): Bir kümeye ait elemanların her birine bir numara atanarak kod okunabilirliğini yükseltmek için tanımlanan **enum** tipleridir.
2. **Referans Tipler** (**reference type**)
 - a) **Gösterici tipler** (**pointer type**)
 - b) **Ara yüz Tipler** (**interface type**)
 - c) **Kendini tanımlayan tipler** (**self-description type**)
 - d) **Diziler** (**array**): Tek boyutlu, çok boyutlu ya da **dizilerin dizisi** (**jagged array**) gibi diziler.
 - e) **Sınıflar** (**class**): **object** ve **string** gibi ön tanımlı sınıflar.

- f) Kullanıcı tanımlı sınıflar (user-defined class)
- g) Doğrudan ulaşımı engellenmiş ya da kutulanmış değer tipler (boxed value types)
- h) Temsilciler (delegate): Yöntemleri referans gösteren tipler.

Derleme ve İcra Süreci

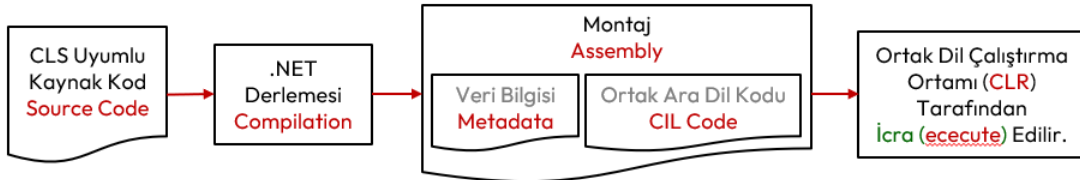
Klasik yazılım geliştirme araçlarıyla derlenen bir uygulama aşağıda gösterildiği gibi bir süreçten geçer. Yani kaynak kod derlenir ve doğrudan işletim sistemi tarafından işlemciye hedef gösterilerek çalıştırılacak şekilde emir kodlarından (instruction code) oluşan çalıştırılabilir bir uygulama elde edilir.



Şekil 5. Klasik Derleme Süreci

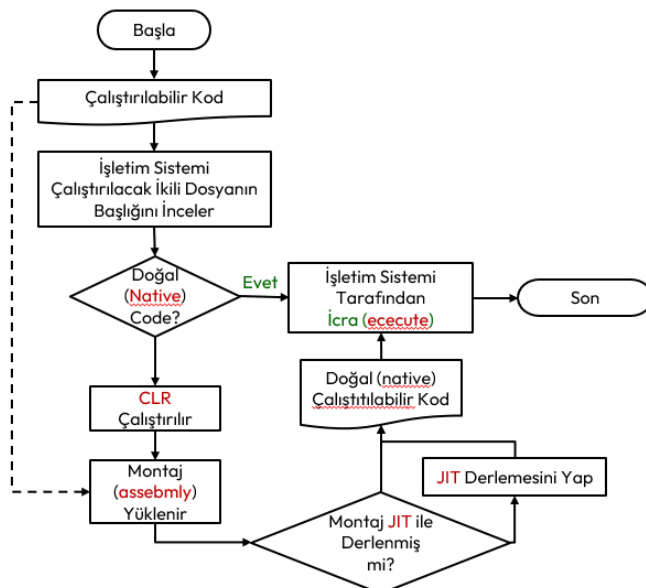
İşlemci doğal olarak ikili sayı sisteminde çalıştığından bu çalıştırılabilir koda ikili-çalıştırılabilir (binary-executable) veya doğal kod (native code) adı verilir.

.NET Framework altyapısında ise kaynak kod derlenir ve ortak dil çalışma ortamı (CLR) tarafından çalıştırılacak şekilde ortak ara dil (CIL) ve veri bilgisinden (metadata) oluşan bir uygulama elde edilir. Derlenmiş bu .Net uygulamasına montaj (assembly) adı verilir.



Şekil 6. .Net Framework Derleme Süreci

Burada derlenecek kaynak kod, ortak dil tarifnamesinde (CLS) belirtilen standartlara uymak zorundadır. CIL kodu ile oluşan bu montaj (assembly) uygulamanın işlemcinin anladığı emir kodları (instruction code) ile hiçbir ilgisi yoktur ve işletim sistemine bağımlı değildir. Bu şekilde yapılan .NET Framework derleme işlemine, yönetilmiş derleme (managed compilation) adı verilir. Burada CLS uyumlu olarak hazırlanmış kaynak kodlara yönetilmiş kod (managed code) adı verilir. CLS uyumlu hazırlanmayan kodlara ise yönetilmemiş kod (unmanaged code) adı verilir. Yönetilmiş kodlar CLR tarafından, yönetilmemiş kodlar ise işletim sistemi tarafından çalıştırılır.



Şekil 7. .Net Altyapısı Derleme ve Çalıştırma Süreci

İşletim sistemi, tarafından bir çalıştırılabilir (executable) kodun icrasını yukarıdaki şekilde yapar. Klasik derleme sonucunda oluşan doğal kod (native code), işletim sistemi tarafından doğrudan belleğe yüklenerek çalıştırılır. .NET Framework de ise, montaj (assembly) uygulamanın çalışması için, ortak dil

çalıştırma ortamının (CLR) işletim sisteminde çalışıyor olması gerekir. Çalışıyor olan CLR, ilgili montaj uygulamasını belleğe yükler ve **ani derleyiciler** (**Just-in-time compiler-JIT compiler**) tarafından derlenerek **doğal kod** (**native code**) elde edilir. Bu doğal kod, CLR kontrolünde, işletim sistemi tarafından çalıştırır. Kısaca **ortak dil altyapısı** (CLI) standartlarına uyumlu olarak çalışan **ortak dil çalıştırma ortamı** (CLR), kurulu olduğu işletim sisteminin kaynaklarını kullanarak, oluşturulan CIL kodunun doğru bir şekilde çalıştırılmasını sağlayan altyapıdır.

Yukarıda anlatılan derlenmiş montaj, hangi işletim sistemine taşınırsa taşınsın, derlenmeden çalıştırılması sağlanmış olur. Bunun için tek gereklilik CLR altyapısının o işletim sisteminde çalışıyor olmasıdır. CLR altyapısı, bir işletim sisteminde çalışan **bileşenin** (**component**) başka bir işletim sistemine problemsiz olarak taşınarak çalışmasını sağlayan bir yapıdır. İşte bu işleme **karşılıklı çalışma** (**interoperability**) adı verilir. Yani uygulamanın işletim sistemine olan bağımlılığını ortadan kaldırmaktadır.

Microsoft tarafından Visual Basic, Visual Java, Visual C++ dillerinde küçük değişiklikler yapılarak CLS uyumlu hale getirilerek .NET dilleri geliştirilmiştir. Böylece yazılımcı takımında dil farklılıklarından doğan geliştirme problemlerine de çözüm bulunmuştur. Böylece yazılımcıların .Net mimarisine geçmek için harcayacağı eğitim ve zamandan tasarruf sağlamıştır.

Ortak dil çalıştırma ortamının montaj uygulamayı belleğe yükleyip, ani derleyicilerden geçirerek işlemcinin çalıştıracığı **doğal kod** (**native code**) elde edilir. Bazı durumlarda içinde bulunan işletim sistemine uygun olarak bu doğal kod önceden oluşturulabilir. Buna **vaktinden önce doğal derleme** (**native ahead-of-time- NAOT**) adı verilir⁹. Bu durum için .net derleyicileri optimize edilmemiştir. Bazı test senaryoları için kullanılabilir.

Ortak Ara Dil Kodu

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemciler de de değişiklikler olmuştur. *İşlemcinin Emri Alma, Çözme ve İcra Döngüsü* başlığı altında belirtilen işlemci kaydedicilere yığın bellek adresini tutan (**stack register**) kaydediciler gibi yeni kaydediciler de eklenmiştir.

Ortak dil tarifnamesinde (CLS) belirtilen standartlara uymak zorunda olan birçok programlama dili bulunmaktadır; VB.NET, C#.NET, VC++.NET, J#.NET, F#.NET, Jscript.NET, WindowsPowerShell, Iron phyton, Iron Ruby. Bu dillerde yazılan kodlar derlenerek **ortak ara dile** (CIL) dönüştürülür.

Fiziki bir işlemci için derlenmiş **doğal kod** (**native code**) dosyaları gibi, .Net altyapısında da **ortak dil kodu** (CIL code), sanki sanal bir işlemci varmış gibi üretilir ve **çalıştırma ortamı** (CLR) tarafından içinde bulunduğu işletim sistemine ait işlemcinin emir kodlarına JIT ile derlenir.

Gerçek işlemcilerde olduğu gibi CIL koduna derlenen her bir uygulama, dört **bellek bölgesinden** (**segment**) oluşur;

1. **Kod bellek** ya da kaynak koda ithafla **metin bellek** (**code segment / text segment**) icra edilecek emirleri içeren kısım.
2. Programın işleyeceği verileri tutan **veri belleği** (**data segment**) kısmı.
3. Yöntemleri **çağırmadan** (**call**) önce mevcut işlemci durumu ve yerel değişkenler gibi bazı işlemleri depolamak için ve fonksiyondan **dönülünce** (**return**) işlemciyi ve çağrı ortamını eski hale getirmek için kullanılan **yığın bellek** (**stack segment**) kısmı. Bu bellek bu nedenle **son giren veri ilk çıkar** (**last in first out-LIFO**) mantığıyla çalışır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan **öbek bellek** (**heap segment**) kısmı.

.NET Framework Kurulumu

Çerçeve yapının son sürümü, <https://dotnet.microsoft.com/en-us/download> adresinden indirilerek kurulur. Kurulum tamamlandıktan sonra konsoldan aşağıdaki komut yazılarak kurulum teyit edilebilir;

⁹ <https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/?tabs=windows%2Cnet8>

```
C:\Users\ILHANOZKAN>dotnet

Usage: dotnet [options]
Usage: dotnet [path-to-application]

Options:
  -h|--help          Display help.
  --info             Display .NET information.
  --list-sdks        Display the installed SDKs.
  --list-runtimes    Display the installed runtimes.

path-to-application:
  The path to an application .dll file to execute.
C:\Users\ILHANOZKAN>
```

Sonrasında PATH değişkenine C# Derleyicisinin aşağıda belirtilen yolu eklenir.

```
C:\Windows\Microsoft.NET\Framework\v4.0.30319\
```

Yerel ortam değişkenlerini Windows bilgisayarda iki şekilde ayarlayabilirsiniz:

Birinci Seçenek;

1-Başlat'a tıklayın, "Hesabınızın ortam değişkenlerini düzenleyin" arayın ve sonuçlarda Denetim Masasını uygulamasını tıklayın.

2-Kullanıcı değişkenlerini değiştirebileceğiniz bir pencere açacaktır.

3-"... için kullanıcı değişkenleri" grubunda "Path" değişkenine çift tıklayarak yukarıda verilen dosya dolunu ekleyebilirsiniz.

İkinci Seçenek;

1-Başlat'a sağ tıklayın ve komut satırına tıklayın.

2-sysdm.cpl yazın ve Tamam'a tıklayın

3-"Gelişmiş" sekmesine tıklayın ve ardından alttaki "Ortam Değişkenleri..." düğmesine tıklayın

4-Bu seçenek, kullanıcı değişkenlerini ve sistem değişkenlerini değiştirebileceğiniz önceki seçenekteki pencereyi açacaktır.

5-"... için kullanıcı değişkenleri" grubunda "Path" değişkenine çift tıklayarak yukarıda verilen dosya dolunu ekleyebilirsiniz.

Böylece kurulum tamamlanmış olur. Konsolda **csc** komutu yazılarak derleyicinin düzgün kurulup kurulmadığı kontrol edilebilir.

```
C:\Users\ILHANOZKAN>csc.exe
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.

This compiler is provided as part of the Microsoft (R) .NET Framework, but only supports
language versions up to C# 5, which is no longer the latest version. For compilers that
support newer versions of the C# programming language, see
http://go.microsoft.com/fwlink/?LinkID=533240

warning CS2008: Hiçbir kaynak dosya belirtilmedi
error CS1562: Kaynağı olmayan çıkışlar için /out seçeneği belirtilmiş olmalıdır

C:\Users\ILHANOZKAN>
```

Daha sonra metin editörü ile ilk kaynak kodumuzu oluşturmak için komut satırında **notepad hello.c** komutu yazılır.

```
C:\Users\ILHANOZKAN>notepad hello.cs
```

Eğer böyle bir dosya yoksa editör bu dosyayı oluşturmak için onay ister. Açılan metin editörü penceresine ilk C programı yazılır ve **hello.cpp** olarak dosyaya kaydedilir.

```
using System;
using System.IO;

namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Aşağıdaki komut ile **hello.cpp** olarak hazırlanan kaynak kodumuz derleyici tarafından icra edilebilir **hello.exe** dosyası olarak derlenir.

```
C:\Users\ILHANOZKAN>csc hello.cs
```

Derlenen programı çalıştırmak için yine aynı komut satırında **hello.exe** yazarak icra edilebilir programımız işletim sistemi tarafında işlemciye hedef gösterilerek çalıştırılır.

```
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
```

```
C:\Users\ILHANOZKAN>
```

Burada programın **notepad** programında yazılması, **csc** programı derlenmesi ile ve derlenen **hello.exe** programının çalışması tarafımızdan yapılmıştır.

Mac OSX ve Linux işletim sistemlerinde de derleyici kurulumu tamamlandıktan sonra terminal yani konsol uygulaması açılarak aşağıdakine benzer şekilde derlemeler yapılabilir.

Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan **kod yazma** (implementation), **derleme** (compile), **icra** (execute) ya da **koşma** (run) ve **izleme** (trace) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, **entegre geliştirme ortamı** (Integrated Development Environment-IDE) olarak adlandırılır. C# programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- **Visual Studio** – Microsoft tarafından geliştirilen, C dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Yalnızca Windows'ta çalışır. Visual Studio, **kod tamamlama** (intellisense), **kullanıcı ara yüzü** (user interfaceUI) hazırlama desteği ve **hata ayıklama** (debugger) ve birçok **eklentisi** (plug-in) olan muazzam özelliklere sahiptir. Bu geliştirme ortamının kurulumu dipnotta verilmiştir.
- **Visual Studio Code** – Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git **sürüm kontrol** (version control) sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- **Xcode** – MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir entegre geliştirme ortamıdır.
- **JetBrains Rider** – Windows, Mac OS X ve Linux'ta .NET ve .NET Core uygulamalarını destekliyor. Rider'ın hızlı performansı, geniş platform ve çalışma zamanı desteği, sürüm kontrol sistemleriyle sorunsuz entegrasyonu ve kapsamlı **derleme çözümleme** (decompile) özellikleri öne çıkan özelliklerinden bazılarıdır.
- **MonoDevelop** – .NET dilleri için oluşturulmuş açık kaynaklı bir geliştirme ortamıdır. Yazılım geliştirme için Mono ve .NET framework'ünü kullanır. MonoDevelop ile MacOSX, Windows ve Linux dahil olmak üzere çeşitli işletim sistemleri için masaüstü ve web uygulamalarını verimli bir şekilde oluşturabilirsiniz.
- **.NET Core CLI** – .NET Core uygulamalarını geliştirmek, oluşturmak, çalıştırmak ve yayınlamak için **komut satırı arayüzüdür** (command line interface). Bu ara yüz adından da anlaşılacağı üzere grafik ara yüzünden yoksundur.

C# ile Nesne Yönelimli Programlama ve Entity Framework: <https://github.com/HoydaBre/csharp>

Entegre geliştirme ortamları metin dosyası olarak tutulan kaynak kodları programcıya renklendirerek gösterir. Bu da programcının kodu anlamasını daha da kolaylaştırır. Ancak kodu dosyaya yine sade metin olarak kaydeder.

Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında online olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- <https://www.tutorialspoint.com/csharp/index.htm>
- https://www.onlinegdb.com/online_csharp_compiler
- <https://onecompiler.com/csharp>

Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı** (**compile time**) denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır.

Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C# dilinin sözdizimi kurallarına uygunluğunu da denetler. Eğer dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir ileti de vermek zorundadır.

```
using System;
using System.IO;
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!")
        }
    }
}
```

Yukarıda hatalı yazılmış bir c programı derlendiğinde aşağıdaki hata alınacaktır.

```
C:\Users\ILHANOZKAN>csc.exe hello.cs
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
This compiler is provided as part of the Microsoft (R) .NET Framework, but only supports
language versions up to C# 5, which is no longer the latest version. For compilers that
support newer versions of the C# programming language, see
http://go.microsoft.com/fwlink/?LinkID=533240
```

```
hello.cs(10,47): error CS1002: ; bekleniyor
```

```
C:\Users\ILHANOZKAN>
```

Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

1. **Derleme zamanı hataları** (**compile time error**): Bu tip hatalar genelde kullanılan dilin **yazım kurallarına** (**syntax**) uyulmadığından, **talimatların** (**statement**) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Koşma zamanı hataları** (**run time error**): Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sifıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.

3. **Mantıksal hatalar** (logical error): Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış **işleçler** (operator) kullanmasından kaynaklanır. Örneğin bir büyüktür (>) işleci yerine küçüktür (<) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken $0! = 1$ yerine $0! = 0$ kabul etmek buna örnek verilebilir.

Yapısal Programlamanın Genel Çerçevesi

Yapısal programlamanın genel çerçevesi aşağıda verilmiştir;

1. Programın başladığı yeri belirten bir **ana fonksiyon** (main function) tanımlanmak zorundadır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Yani iş sürecini sağlayacak programlama fonksiyonların birbirini çağırmasıyla yapılır.
2. Her fonksiyonda; İlk önce değişken, dizi ve yapı gibi **veri yapıları** (data structure) tanımlanır. Daha sonra bu veri yapılarını işleyecek **if, if-else, switch, while, for, do-while, continue, break, goto** ve **return** gibi **kontrol yapıları** (control structure) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe **talimatlardan** (statement) oluşabilir.

Yapısal programlamaya örnek olarak aşağıdaki C kodu örnek verilebilir;

```
//Koda Dahil Edilen Başlık
#include<stdio.h>

//Fonksiyon Prototipleri:
void diziOku(int pDizi[],int pUzunluk);
void diziYaz(int pDizi[],int pUzunluk);
float diziOrtalama(int pDizi[],int pUzunluk);

//Ana Fonksiyon
int main() {
    //Burada yapılmak istenen iş süreci fonksiyonlar ardımıyla yapılır.
    int dizi[5]; // Ana fonksiyonda da veri yapısı tanımlanabilir.
    float ortalama;
    diziOku(dizi,5);
    diziYaz(dizi,5);
    ortalama=diziOrtalama(dizi,5);
    printf("Dizi Ortalaması: %.2f",ortalama);
    return 0;
}

//Fonksiyonların (Prototipleri Yukarıda Verilmiş) Tanımlamaları:
void diziOku(int pDizi[],int pUzunluk){
    int sayac; // Veri yapısı tanımlandı
    printf("%d Elemanlı Dizi Okunacaktır:",pUzunluk);
    for (sayac=0; sayac < pUzunluk; sayac++) //Kontrol Yapısı
        scanf("%d",&pDizi[sayac]);
}

void diziYaz(int pDizi[],int pUzunluk){
    int sayac; // Veri yapısı
    printf("%d Elemanlı Dizi:\n",pUzunluk);
    for (sayac=0; sayac < pUzunluk; sayac++) // Kontrol Yapısı
        printf("%4d",pDizi[sayac]);
    printf("\n");
}

float diziOrtalama(int pDizi[],int pUzunluk){
    int sayac; // Veri yapıları
    float toplam=0;
    for (sayac=0; sayac < pUzunluk; sayac++) //Kontrol Yapısı
        toplam+=pDizi[sayac];
    return toplam/pUzunluk; //Kontrol Yapısı
```

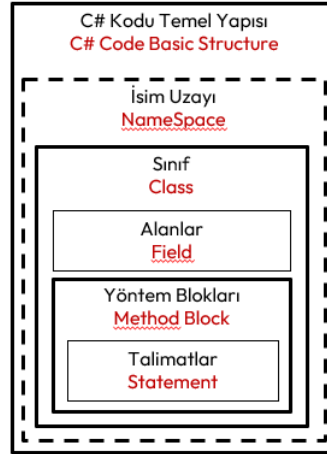

}

Nesne Yönelimli Programın Genel Çerçevesi

C# dili, Java dili gibi saf nesne yönelimli bir dildir. Nesne olmadan program yazılamaz. Nesne yönelimli programlamada;

- Nesneler, **durum** (state) ve **davranışlara** (behavior) sahiptir.
- Kodlaması yapılacak iş, nesnelerin birbirlerine **ileti göndermesiyle** (message-passing) yapılır.

Nesnelerin hangi durum ve davranışa sahip olacağı programcı tarafından kullanıcı tanımlı **sınıflarla** (class) belirlenir. Bu nedenle her c# programı en az bir sınıf içerir.



Şekil 8. C# Kodu Temel Yapısı

Kodun genel yapısı aşağıda verilmiştir;

```

1  using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
2
3  namespace ÖrnekUygulama // ÖrnekUygulama adında bir isim Uzayı
4  { // Blok Başı
5      class Sınıf // "Sınıf" adında bir Sınıf.
6      {
7          // Alan Değişkenleri (fields):
8          int yaş = 5;
9          // Özellikler (properties):
10         public int Yaş
11         {
12             get { return yaş; }
13             set { yaş = value; }
14         }
15         public string Adi { get; set; }
16
17         // Metotlar (methods):
18         public void BilgileriYazdırYöntemi()
19         {
20             Console.WriteLine($"Yaş: {Yaş}, Adı: {Adi}"); //Talimat (statement)
21         }
22     }
23 } // Blok Sonu

```

Bir **sınıf** (class) her zaman bir **namespace** ile belirtilen bir isim uzayında olmak zorunda değildir. Sınıflar **alan değişkenleri** (field), **özellikler** (property) ve **yöntemler** (method) içerebilirler.

Yapısal programlamada ana fonksiyon ile program başlar. Nesne yönelimli programlamada ise programın başladığı yer için bir sınıfta tanımlanan **static void Main(string[] args)** yöntemi eklenir. Böylece derleme sonrasında **icra edilebilir** (executable) bir **uygulama** (application) ortaya çıkar. Bu yöntemin olmadığı kodlar **sınıf kütüphanesi** (class library) olarak adlandırılır.

Bir proje birden fazla kod dosyasından oluşabilir. Her bir sınıf ayrı bir dosyada yer alabilir. Tüm projeyi oluşturan bu kodlarının tamamına **kod tabanı** (codebase) adı verilir.

Aşağıda yukarıdaki sınıftan nesne imal edip kullanan bir konsol uygulaması vermiştir;

```
using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
namespace ÖrnekUygulama // ÖrnekUygulama adında bir isim Uzayı
{
    class Sınıf // "Sınıf" adında bir sınıf
    {
        // Alan Değişkenleri (fields):
        int yaş = 5;
        // Özellikler (properties):
        public int Yaş
        {
            get { return yaş; }
            set { yaş = value; }
        }
        public string Adi { get; set; }

        // Metotlar (methods):
        public void BilgileriYazdırYöntemi()
        {
            Console.WriteLine($"Yaş: {Yaş}, Adı: {Adi}");
        }
    }
    class Program // Programı başlatacak Main yöntemini içeren sınıf
    {
        static void Main(string[] args) // Programın başlangıç noktası
        {
            Sınıf nesne = new Sınıf(); // Sınıftan "nesne" adlı nesne imal edildi
            nesne.Yaş = 10; // nesnenin Yaş özelliğine değer atama
            nesne.Adi = "Ali"; // nesnenin Adi özelliğine değer atama
            nesne.BilgileriYazdır(); // nesnenin yöntemini çağırarak nesneye ileti-gönderme
        }
    }
}
```

C# Projesi Oluşturma ve Çalıştırma

Visual Studio 2022 ile Proje Oluşturma

Programda sırasıyla şu adımlar takip edilir: *Visual Studio* açılır;

1. Programı ilk defa çalıştırıyorsanız, karşılama sayfasından *Create a new project* seçiniz. Eğer program açık ise *File* menüsüne giderek *File → New Project* seçiniz.
2. Proje tipi olarak *Console Application* seçilir.
3. *Project Name*, *Location* ve *Solution Name* belirlenerek ilk proje oluşturulur.
4. Projeye oluşturma sihirbazının *Additional Information* sayfasında *Do not use top-level statements* kutucuğu ön tanımlı olarak **ŞEÇİLİ** değildir. Bu durumda projemiz yeni proje şablonuyla oluşur.
5. *Solution Explorer* üzerinde *Program.cs* dosyası açılır.

.NET 6'dan başlayarak, yeni proje şablonuyla oluşan C# konsol uygulamaları için **Program.cs** dosyası aşağıdaki şekilde oluşur;

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Yeni proje şablonu ile oluşturulan projelerde **Program.cs** kodunun başında aşağıdaki açıklama her zaman bulunur;

```
// See https://aka.ms/new-console-template for more information
```

Eğer eski proje şablonuyla proje oluşturulmak istenirse yukarıda anlatılan dördüncü adımda projeye oluşturma sihirbazının *Additional Information* sayfasında *Do not use top-level statemets* kutucuğu **ŞEÇİLİ** bırakılır. Bu durumda **Program.cs** alışık olduğumuz eski şekilde oluşur;

```
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Aslında ilk oluşan yeni tip **Program.cs** dosyasında yazılan kodlar, ikinci oluşan eski tip **Program.cs** dosyasındaki **static void Main(string[] args)** yönteminin içerisinde yazılıyor gibi kabul edebilirsiniz. Bu proje şablonuyla oluşturulan **Program.cs** dosyasına konsola bir şey yazmak ve konsoldan bir şey okumak için aşağıdaki yönergeler uygulamaya örtük olarak eklenir¹⁰:

```
- using System;
- using System.IO;
- using System.Collections.Generic;
- using System.Linq;
- using System.Net.Http;
- using System.Threading;
- using System.Threading.Tasks;
```

6. Menüde, Hata ayıklama için *Debug* → *Start Debugging* tıklanır veya F5 tuşuna basılarak hata ayıklama modunda program çalıştırılır. Ya da hata ayıklamadan derleme için *Start Without Debugging* tıklanarak veya Ctrl + F5 tuşlarına basarak program doğrudan çalıştırılabilir.

.NET Core CLI ile Proje Oluşturma

Konsol uygulamasında sırasıyla şu adımlar takip edilir:

1. Bil klasör oluşturulur (**C:\Users\ilhan>mkdir hello_world**)
2. Ardından oluşturulan klasöre girilir (**C:\Users\ilhan>cd hello_world**).
3. Yeni bir proje oluşturmak için **dotnet new console** komutu çalıştırılır.
4. İki adet dosya oluşacaktır;

Biri proje dosyası (**hello_world.csproj**);

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

Diğeri program **Program.cs** dosyası;

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Program eski şekilde oluşturulmak istenirse **dotnet new console --use-program-main** komutu ile proje oluşturulur. Bu durumda **Program.cs** alışık olduğumuz şekilde oluşur.

5. Gerekli olan paketlerin projeye dahil edilmesi için **dotnet restore** komutu kullanılır.
6. Derleme **Hata Ayıklama** (**debug**) için yapılacak ise **dotnet build** veya

¹⁰ <https://aka.ms/new-console-template>

7. Derleme **Yayım** (**release**) için yapılacak **dotnet build -c Release** ile yapılır.
8. Derleme sonrası oluşan uygulamaya çalıştırılacak ise **dotnet run** komutu yazılarak yapılır.

```
C:\Users\ilhan>mkdir hello_world

C:\Users\ilhan>cd hello_world

C:\Users\ilhan\hello_world>dotnet new console
"Konsol Uygulaması" şablonu başarıyla oluşturuldu.

Oluşturma sonrası eylemleri işleniyor...
C:\Users\ilhan\hello_world\hello_world.csproj geri yükleniyor:
Geri yükleme başarılı oldu.

C:\Users\ilhan\hello_world>dotnet restore
Geri yükleme tamamlandı (0,3sn)

"0,6" sn'de başarılı oluşturun

C:\Users\ilhan\hello_world>dotnet run
Hello, World!

C:\Users\ilhan\hello_world>
```